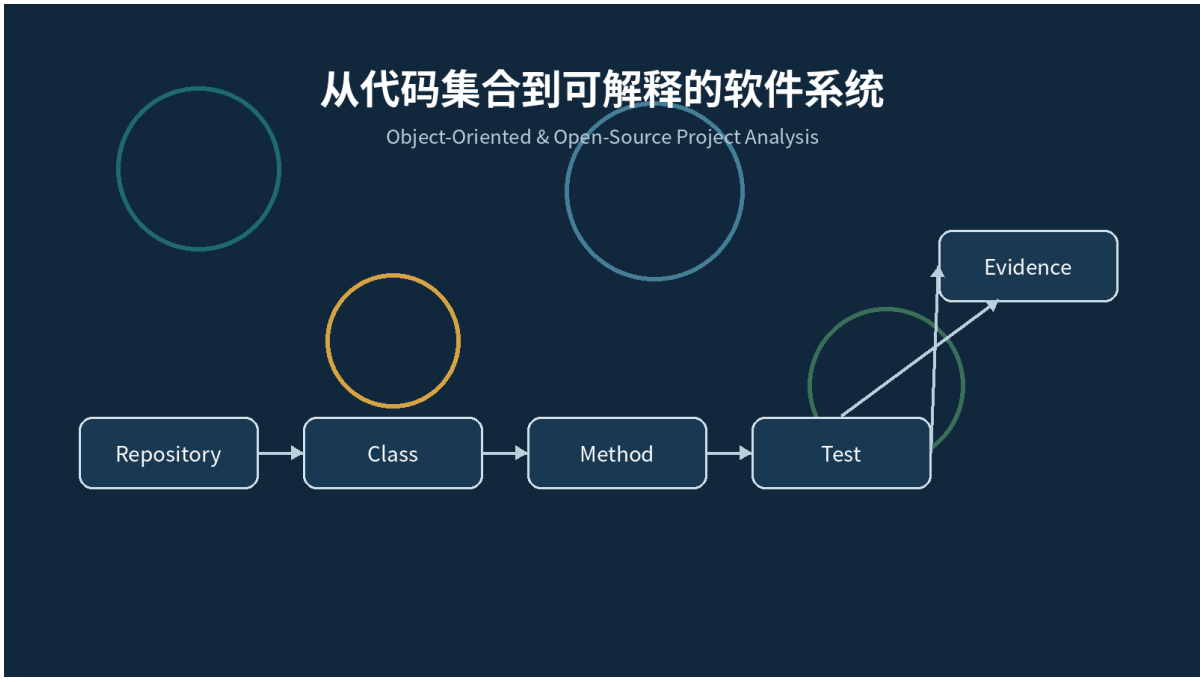
面向对象与开源项目分析

AI 辅助代码理解 · 开源项目结构分析



图文实训教材 · 教师/学生通用

案例：Spring PetClinic | Java · Spring Boot · GitHub

复旦大学《软件分析与测试》课程讲义

2026.9

目录

教材使用说明 5
适用课程 5
教材组织方式 5
项目导入 从“会写类”到“会读系统” 7
学习目标 7
案例项目：Spring PetClinic 7
实训环境 8
实训 1 仓库侦察：建立项目全景 9
1.1 实训目标 9
1.2 获取项目并运行 9
1.3 根目录快速扫描 10
1.4 AI 第一次介入：只让它“画地图” 11
1.5 学生操作记录 11
实训 2 识别技术栈与启动链 12
2.1 为什么要先找到启动入口 12
2.2 从构建文件识别技术栈 12
2.3 找到 PetClinicApplication 12
2.4 防止“常见架构幻觉” 13
2.5 思考题 13
实训 3 从包结构进入领域对象 14
3.1 从业务名词开始，而不是从最长的类开始 14
3.2 从父类看共性 14
3.3 四种关系的判别 15
3.4 AI 辅助类分析：五问法 15
3.5 类责任卡 16
实训 4 追踪一个真实业务调用链 17
4.1 从一个用户动作开始 17

4.2 找到 Web 入口 17
4.3 用“入口—数据—判断—调用—返回”五步法阅读方法 17
4.4 调试与静态追踪结合 18
4.5 AI 调用链提示词 18
4.6 调用链证据表 18
实训 5 AI 辅助代码理解方法 19
5.1 AI 最适合做什么 19
5.2 四类高价值提问 19
5.3 低质量提示词与改写 20
5.4 AI 答案证据分级 20
5.5 AI 使用日志 20
实训 6 用测试验证代码理解 22
6.1 为什么测试是“可执行说明书” 22
6.2 测试阅读四步法 22
6.3 从成功路径读到失败路径 22
6.4 设计一个新增测试 23
实训 7 GitHub 代码导航与 Issue 分析 24
7.1 定义与引用：比全文搜索更可靠 24
7.2 从真实 Issue 进入维护场景 24
7.3 Issue 分析卡 25
7.4 拓展：读一次代码历史 25
综合实训 输出项目理解报告 26
8.1 综合任务说明 26
8.2 必交成果 26
8.3 项目理解报告结构 26
8.4 评分量表 27
8.5 口头答辩问题 27
附录 A AI 提示词卡 29
附录 B 学生实训记录模板 31
B.1 仓库观察表 31

B.2 核心类卡 31
B.3 AI 证据矩阵 31
B.4 调用链记录 32
B.5 实训反思 32
附录 C 教师参考与教学组织建议 34
C.1 教学重点 34
C.2 课堂节奏建议（8 学时版本） 34
C.3 常见学生误区与纠正 34
C.4 教师参考答案（核心方向） 35
参考资料与版本说明 36
开源与教学使用说明 36
结语 36

教材使用说明

本教材面向已经学习过 Java 基础与面向对象概念、但尚未形成“大型代码阅读”能力的学生。教学重点不是从零编写一个新系统，而是借助真实开源项目，训练学生在有限时间内识别项目结构、核心对象、业务调用链与验证证据。

教材将 AI 定位为“代码理解搭档”，而不是答案来源。学生可以让 AI 帮助建立地图、提出假设、解释局部代码和比较备选设计，但重要结论必须回到源码、配置、测试、运行结果或官方文档进行核验。

建议学时

基础版 8 学时；完整课堂版 12 学时；课程大作业版可扩展至 16 学时。每个实训单元均可独立布置，也可按本教材顺序形成完整项目。

适用课程

- 面向对象程序设计（Java）
- 软件工程 / 软件构造
- 开源软件分析
- 程序理解与软件维护
- AI 辅助软件工程实践

教材组织方式

模块	学生要回答的问题	主要产出
项目导入	为什么真实项目比课堂小程序难读？	能力地图
实训 1 仓库侦察	这个项目由哪些部分组成？	项目结构图
实训 2 启动与技术栈	系统从哪里开始运行？	启动链
实训 3 面向对象关系	核心对象及其关系是什么？	领域类图
实训 4 调用链	一个业务是怎样执行的？	动态调用链
实训 5 AI 辅助理解	怎样问 AI 才能得到可验证答案？	提示词与证据表

模块	学生要回答的问题	主要产出
实训 6 测试验证	怎样证明自己的理解没有偏？	测试解释记录
实训 7 Issue 分析	怎样从真实问题反推代码？	Issue 分析卡
综合实训	怎样把理解压缩成可复用知识？	项目理解报告

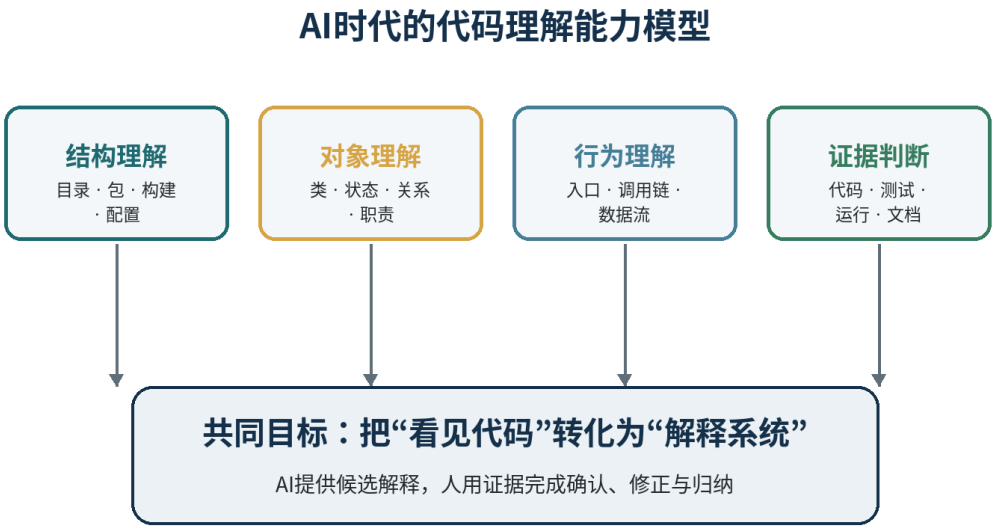


图 0-1 AI 时代的代码理解能力模型

版本说明

案例按 2026 年 9 月可访问的 Spring PetClinic 主仓库与 GitHub 官方代码导航文档编写。开源项目会持续变化，课堂使用时应优先以当前仓库代码为准。

项目导入 从“会写类”到“会读系统”

在入门阶段，我们常用 Animal、Student、BankAccount 等小程序理解类与对象。但真实软件项目的困难并不在于某一行 Java 语法，而在于：文件多、依赖多、对象多、入口多，而且很多行为由框架在运行时连接起来。学生第一次进入一个成熟项目时，最容易陷入“逐文件阅读”，读了很多却没有形成整体模型。

因此，本教材把代码理解拆成四类能力：结构理解、对象理解、行为理解和证据判断。结构理解回答“代码放在哪里”；对象理解回答“谁负责什么”；行为理解回答“程序怎样动起来”；证据判断回答“我凭什么相信这个解释”。

任务情境

你刚加入一个维护 Spring Boot 系统的小组。组长没有让你立即开发功能，而是要求你在半天内回答：项目入口在哪里？Owner 模块如何工作？新增 Owner 时经过哪些类？如果 AI 的解释与代码冲突，应当相信谁？

学习目标

1. 能在 15 分钟内完成陌生 Java 仓库的第一轮结构侦察。
2. 能识别核心领域对象，并区分继承、关联、依赖和调用。
3. 能以一个用户操作为线索追踪 Controller、验证、Repository 和数据持久化。
4. 能使用 AI 生成候选解释，并用源代码、测试和运行结果验证。
5. 能输出项目地图、领域类图、调用链图和证据矩阵。

案例项目：Spring PetClinic

Spring PetClinic 是 Spring 社区长期维护的示例应用。官方主仓库当前说明其为 Spring Boot 应用，支持 Maven 或 Gradle 构建，构建要求 Java 17 或更高版本；默认配置可使用内存型 H2 数据库启动。它包含 Owner、Pet、Vet、Visit 等典型业务对象，非常适合作为面向对象与项目级代码理解的练习对象。

项目要素	本教材关注点
README.md	项目用途、启动方式、数据库和 IDE 入口
pom.xml / build.gradle	技术栈、依赖和构建
PetClinicApplication	应用启动入口

项目要素	本教材关注点
owner / vet / model / system	领域与公共模型
src/main/resources	模板、静态资源、配置
src/test	行为证据与回归验证

实训环境

工具	用途	最低要求
JDK	编译与运行 Java	17+
IntelliJ IDEA / VS Code / Eclipse	代码导航、断点、运行	任选一种
Git	获取与更新仓库	能执行 clone / pull
Maven Wrapper / Gradle Wrapper	构建与运行	优先使用项目自带脚本
浏览器	访问本地系统与 GitHub	现代浏览器
AI 助手	解释、归纳、对比、生成候选假设	支持粘贴代码或项目上下文

关键原则

面对陌生项目时，先建立“地图”，再进入“街道”，最后进入“房间”。也就是：先仓库结构，再领域对象，再局部方法。

实训 1 仓库侦察：建立项目全景

1.1 实训目标

本实训只做一件事：在不深入任何业务方法的前提下，为项目建立第一张地图。你需要知道哪些文件值得优先看，以及哪些内容此刻可以暂时忽略。



图 1-1 仓库侦察路线

1.2 获取项目并运行

```
git clone https://github.com/spring-projects/spring-petclinic.git
cd spring-petclinic
./mvnw spring-boot:run
```

Windows PowerShell 下可使用 mvnw.cmd。若选择 Gradle，则使用项目自带的 gradlew/gradlew.bat。项目成功启动后，在浏览器访问本地 8080 端口。

教师提示

第一次运行的目的不是排查所有依赖问题，而是建立“代码—运行结果”的连接。若环境安装耗时，可由教师提供已配置环境，避免实训目标被环境问题淹没。

1.3 根目录快速扫描

只看根目录，先不要打开 Java 文件。根据文件名推断项目有哪些组成部分，然后再用 README 与构建文件核对。



图 1-2 Spring PetClinic 第一张项目地图

观察对象	先问什么	不要急着做什么
README.md	项目是什么？怎样启动？	逐段背诵文档
pom.xml / build.gradle	使用什么框架和依赖？	研究每一个版本号
src/main/java	业务代码按什么方式组织？	逐文件阅读
src/main/resources	模板、静态资源和配置在哪？	修改配置
src/test	哪些行为被自动验证？	一开始就读完所有测试
.github	项目如何协作和自动化？	研究全部 CI 细节

1.4 AI 第一次介入：只让它“画地图”

推荐提示词

这是一个 Java Spring Boot 开源项目。暂时不要逐行解释代码。请先从项目结构角度分析：①哪些目录最值得优先阅读；②哪些文件决定项目如何启动；③哪些目录属于业务代码、配置、页面和测试；④如果只有 30 分钟，建议按什么顺序阅读。每个判断都要说明需要通过哪些文件验证。

这个提示词有两个设计点。第一，限定任务为“结构分析”，阻止 AI 一开始就泛化到所有代码；第二，要求“说明验证位置”，把答案从自然语言解释转化为可检查的假设。

1.5 学生操作记录

序号	你的初步判断	验证文件/目录	验证结果
1	项目采用 _____ 构建	_____	□正确 □修正
2	应用入口可能是 _____	_____	□正确 □修正
3	Owner 相关代码位于 _____	_____	□正确 □修正
4	页面模板位于 _____	_____	□正确 □修正
5	测试代码位于 _____	_____	□正确 □修正

阶段验收

完成后，你应该能在不打开 90% 文件的情况下，用 2 分钟口头说明“这个仓库大致由哪些部分组成”。如果还只能列举文件名，说明尚未形成结构模型。

实训 2 识别技术栈与启动链

2.1 为什么要先找到启动入口

一个 Web 项目有很多类，但系统总需要某个入口把运行环境组织起来。找到入口并不等于理解所有框架细节，却能回答一个关键问题：程序从哪里开始，以及哪些组件随后被框架接管。

2.2 从构建文件识别技术栈

在当前主仓库中，pom.xml 指定 Spring Boot 父项目并设置 Java 版本。你不需要记住全部依赖，而要学会把依赖按职责归类：Web、数据访问、模板、验证、测试、数据库、开发辅助。

依赖类别	你要理解的意义	典型问题
Web / MVC	如何接收请求并返回页面	Controller 在哪里？
Data JPA	对象如何映射并访问数据库	Repository 是什么？
Thymeleaf	页面模板如何渲染	模板在哪里？
Validation	输入如何被校验	错误如何返回表单？
Test	行为如何自动验证	哪些边界条件被覆盖？

2.3 找到 PetClinicApplication

官方 README 将 PetClinicApplication 标为主类。当前主类使用 Spring Boot 应用注解，并在 main 方法中启动 SpringApplication。对于实训来说，不需要背诵注解原理，但必须能解释：main() 把控制权交给 Spring Boot，后者创建应用上下文、扫描组件并启动 Web 环境。

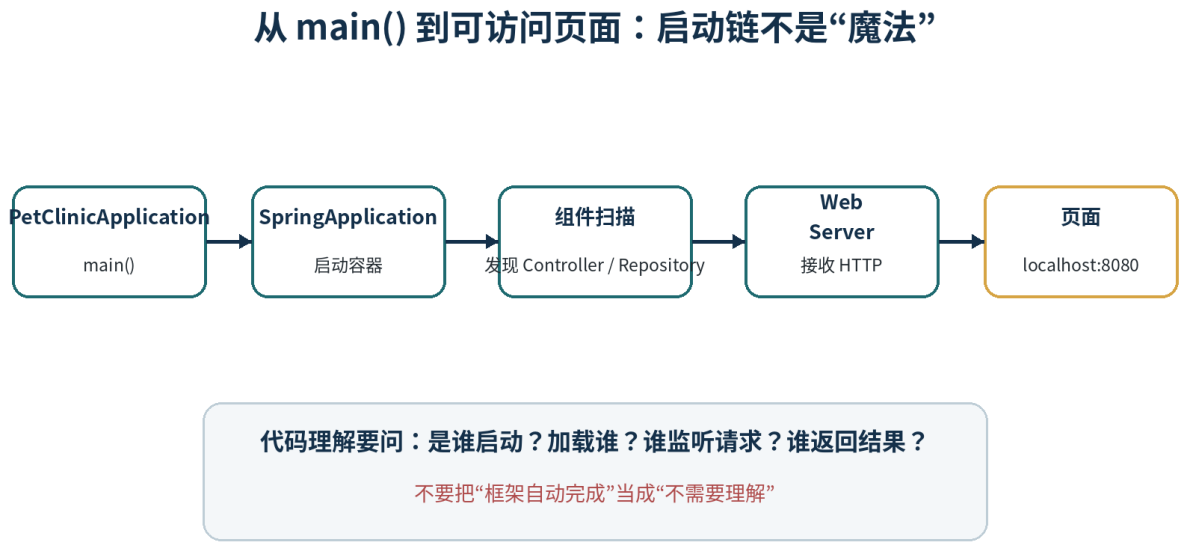


图 2-1 从 main() 到页面的启动链

2.4 防止“常见架构幻觉”

AI 在看到 Spring Boot 项目后，常会自动给出“Controller → Service → Repository → Database”的标准分层图。这个图在很多项目中成立，但不等于当前项目必然存在独立 Service 层。学生应当搜索 service 包、Service 类和 Controller 的实际依赖，而不是把通用模板当成仓库事实。

核验任务

问 AI：“请总结这个项目的完整分层架构。”然后在仓库中核查它是否声称存在独立 Service 层。把 AI 的原话、搜索证据和你的修正写进记录表。

AI 声称	代码证据	结论
_____	_____	□确认 □否定 □部分成立
_____	_____	□确认 □否定 □部分成立

2.5 思考题

1. 框架自动扫描组件，是否意味着开发者不需要理解组件之间的依赖？为什么？
2. 如果应用可以正常启动，但某个页面访问报错，启动链和业务调用链中哪一部分更值得优先排查？
3. 构建文件中的依赖很多，为什么第一轮只做“分类”而不是逐个研究？

实训 3 从包结构进入领域对象

3.1 从业务名词开始，而不是从最长的类开始

面向对象项目最值得优先理解的是“领域对象”。一个简单但有效的方法是名词法：把页面、README、包名和类名中的业务名词列出来，再判断哪些名词被建模成类。PetClinic 中常见的业务名词包括 Owner、Pet、Vet、Visit、PetType、Specialty。

操作

浏览 owner、vet、model 等包，只记录“业务名词”和“职责猜测”，暂时不要展开每个方法。5 分钟后，将这些名词按照“人、动物、事件、类型/分类”重新分组。

3.2 从父类看共性

真实项目中的继承往往用于表达共同属性和共同身份，而不是为了展示语法。PetClinic 的公共模型中可以看到 BaseEntity、NamedEntity、Person 等抽象层次：id 属于实体的共同身份；name 适用于某些“有名称”的实体；firstName 和 lastName 则属于人的共同信息。

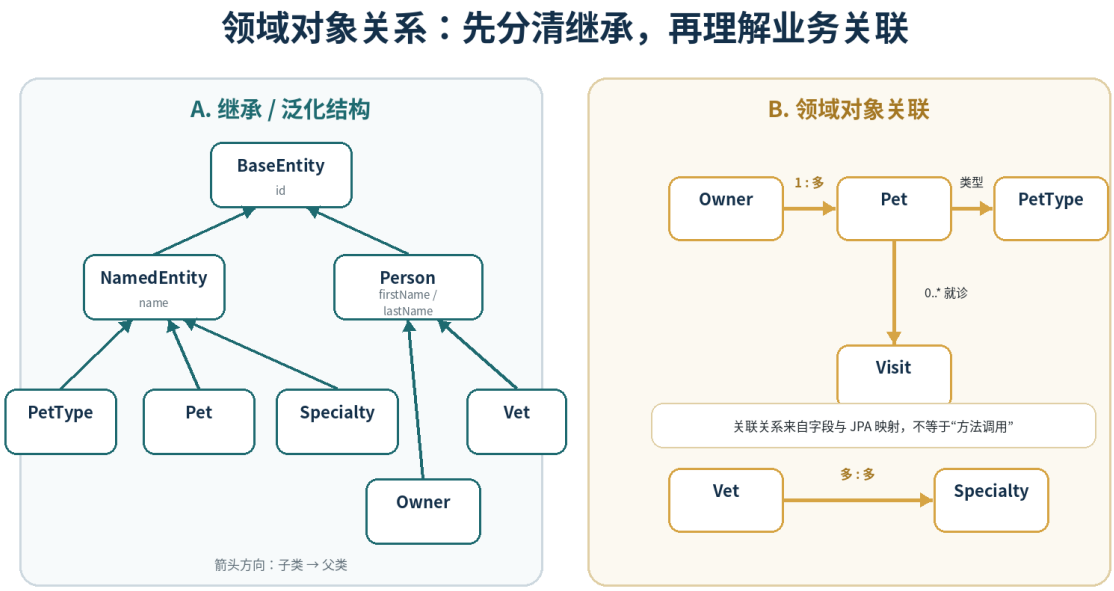


图 3-1 领域对象关系示意图（用于训练抽象，不等同于数据库 ER 图）

3.3 四种关系的判别



图 3-2 四类常见对象关系

判断关系时，不要看到两个类名同时出现就画箭头。继承关注“类型是否属于另一类型”；包含/关联关注“对象是否持有另一个对象或集合”；依赖关注“完成职责时是否需要协作者”；调用则发生在具体执行路径中。

观察证据	更可能对应的关系	示例思路
extends / implements	继承/实现	子类复用并扩展父类抽象
字段为另一个实体或集合	关联/聚合	Owner 持有 Pet 集合
构造器参数为某接口	依赖	Controller 依赖 Repository
方法内部执行 another.method()	调用	保存、查询等动态行为

3.4 AI 辅助类分析：五问法

推荐提示词

请从五个角度分析这个 Java 类：①它代表现实业务中的什么对象或责任；②保存什么状态；③能执行什么行为；④与哪些类存在继承、关联、组合或依赖；⑤如果删除这个类，哪些功能会受到影响。每个结论请指出代码证据；证据不足时写“待验证”。

3.5 类责任卡

类	一句话责任	关键状态	关键关系	证据位置
Owner	_____	_____	_____	_____
Pet	_____	_____	_____	_____
Vet	_____	_____	_____	_____
Visit	_____	_____	_____	_____

阶段验收

脱离代码后，能否用 90 秒解释 Owner、Pet、Visit 三者的关系，并说明哪些属于静态对象关系、哪些属于动态方法调用？

实训 4 追踪一个真实业务调用链

4.1 从一个用户动作开始

大型项目无法一次“读完”。更有效的方法是选择一个可以被用户观察的业务动作，然后沿着执行路径向下追踪。本实训选择“新增 Owner”：用户打开表单、填写数据、提交，系统校验、保存，并跳转到 Owner 详情页。

4.2 找到 Web 入口

当前 OwnerController 中同时存在用于显示新增表单的 GET 映射和用于处理表单提交的 POST 映射；Controller 通过构造器接收 OwnerRepository。由此可以得到一个重要事实：至少在这条业务链上，Controller 可以直接调用 Repository，不应凭习惯强行补出一个不存在的 Service。

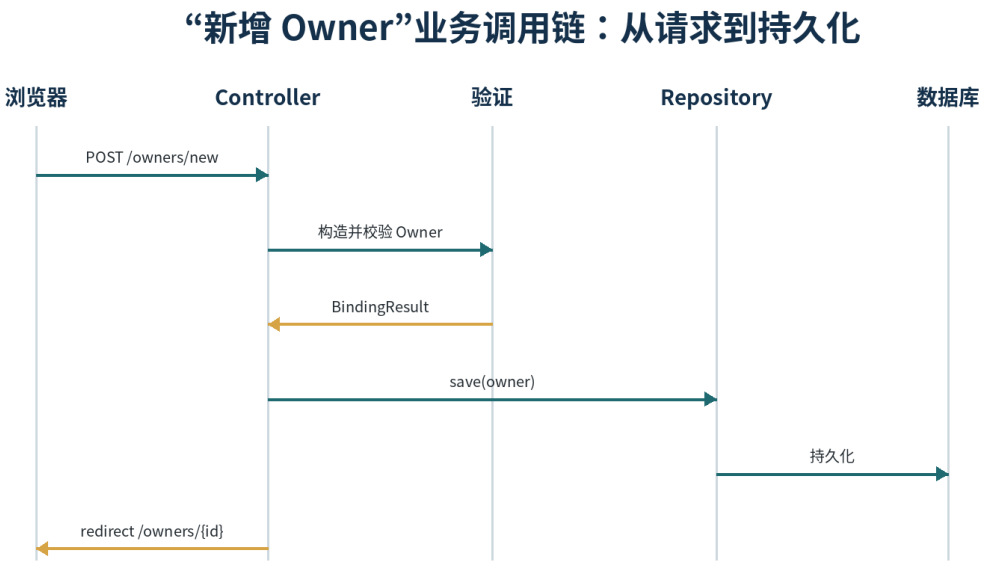


图 4-1 “新增 Owner”动态调用链

4.3 用“入口—数据—判断—调用—返回”五步法阅读方法

步骤	阅读问题	记录示例
入口	什么请求/事件进入这个方法？	POST /owners/new
数据	输入如何形成对象？	表单字段绑定为 Owner
判断	有哪些校验或分支？	验证失败则返回表单

步骤	阅读问题	记录示例
调用	向下调用谁？	Repository 保存 Owner
返回	结果如何反馈给用户？	重定向到详情页

4.4 调试与静态追踪结合

静态阅读告诉我们“代码可能怎样走”，断点和运行则告诉我们“这一次实际怎样走”。建议在处理新增 Owner 的方法入口、保存调用和详情页显示入口处设置断点，观察 Owner 对象中的 id 何时从空值变为已生成值。

观察重点

不要把断点当作“看每一行”。重点观察对象状态如何变化：提交前有哪些字段？验证失败时哪些字段有错误？保存后 id 发生了什么变化？重定向后系统如何重新加载 Owner？

4.5 AI 调用链提示词

推荐提示词

我要理解“新增 Owner”业务。请从 HTTP 请求开始，按实际执行顺序分析：请求 → 参数绑定 → Controller → 数据验证 → Repository → 持久化 → 重定向。每一步都给出类名、方法名和文件位置；如果无法从现有上下文确认，请写“需要进一步验证”，不要推测。

4.6 调用链证据表

步骤	类/文件	方法或证据	你观察到的对象状态
请求入口	_____	_____	_____
校验	_____	_____	_____
保存	_____	_____	_____
跳转与详情加载	_____	_____	_____

实训 5 AI 辅助代码理解方法

5.1 AI 最适合做什么

AI 适合把大量局部信息压缩为候选结构，例如总结一个类的职责、比较两个实现、解释陌生注解、根据给定代码生成调用链草图、把测试用例归纳为行为边界。但 AI 对当前仓库的真实结构可能掌握不完整，也可能把常见模式错误套用到当前项目。

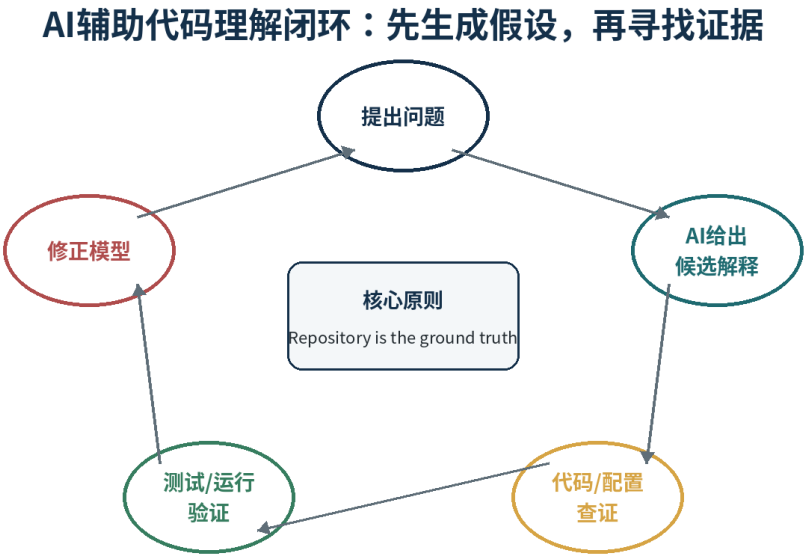


图 5-1 AI 辅助代码理解的证据闭环

5.2 四类高价值提问

提问类型	目的	典型句式
地图型	快速建立全局	“只分析目录与职责，不进入方法细节。”
追踪型	建立动态路径	“从这个 HTTP 入口按执行顺序追踪。”
比较型	发现设计选择	“比较直接 Repository 调用与增加 Service 层的差异。”
反证型	主动找错	“指出你刚才解释中最可能不成立的三点，并给出验证方法。”

5.3 低质量提示词与改写

低质量问法	问题	改写
“解释这个项目。”	范围过大，容易得到套话	限定模块、目标和证据
“这个类是干什么的？”	容易停留在类名复述	要求状态、行为、关系、删除影响
“画完整调用链。”	上下文不足时易编造	限定一个用户动作并允许写待验证
“有哪些设计模式？”	容易“看什么都像模式”	要求参与类、代码证据、解决问题

5.4 AI 答案证据分级

等级	证据来源	使用原则
A	实际运行、断点、自动测试	优先用于确认动态行为
B	当前仓库源码与配置	确认静态结构和实现细节
C	当前版本官方 README/官方文档	确认使用方式和框架说明
D	AI 自然语言解释	仅作假设，必须继续核验

原则

AI 答案不是“错误”才需要验证，而是“无论看起来多合理都需要验证”。专业代码理解与普通问答的差别，就在于你能指出依据。

5.5 AI 使用日志

问题	AI 核心结论	证据	最终判断	修正后的模型
_____	_____	_____	□确认 □修正	_____
_____	_____	_____	□确认 □修正	_____

问题	AI 核心结论	证据	最终判断	修正后的模型
_____	_____	_____	□确认 □修正	_____

实训 6 用测试验证代码理解

6.1 为什么测试是“可执行说明书”

源码展示“怎样实现”，测试则表达“在什么条件下应该出现什么结果”。对陌生项目来说，测试能够迅速暴露作者最在意的输入、输出、错误条件和回归边界。当前 OwnerControllerTests 会构造 Owner、Pet、Visit 等对象，并通过 MockMvc 验证创建、查找、更新、显示以及错误输入等场景。

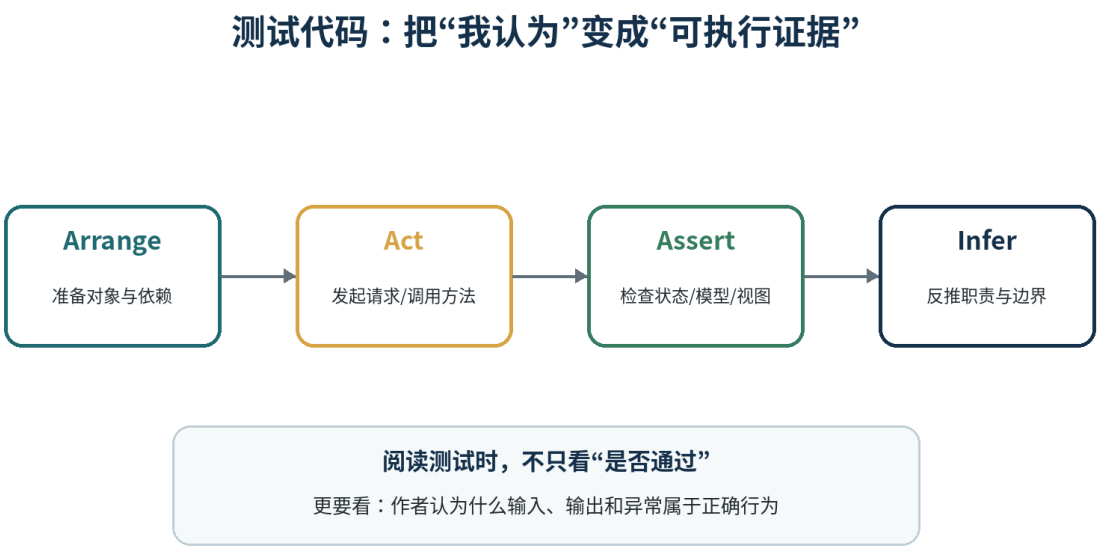


图 6-1 用 Arrange—Act—Assert 反推程序职责

6.2 测试阅读四步法

1. Arrange：测试准备了哪些对象、模拟了哪些依赖？
2. Act：测试实际发起了哪个请求或方法调用？
3. Assert：测试检查了状态码、模型、视图还是对象状态？
4. Infer：这些断言说明被测代码承担了什么职责、哪些边界不能被破坏？

6.3 从成功路径读到失败路径

只读“成功创建 Owner”的测试，会让你误以为业务逻辑只有保存。继续阅读缺少地址、电话等字段时的错误测试，才能理解验证机制和失败页面也是系统行为的一部分。软件维护中，失败路径往往比成功路径更容易在修改中被破坏。

测试阅读提示词

请阅读这个测试类，不要逐行翻译。请分别说明：测试对象是谁；每个测试建立什么场景；输入条件是什么；期望结果是什么；这些测试反映了被测类的哪些职责。最后总结：如果我要修改被测试类，最需要防止破坏哪些行为？

6.4 设计一个新增测试

请在不修改业务代码的情况下，先设计一个你认为值得新增的边界测试。例如：姓名字段异常、URL 中 ownerId 与表单对象 id 不一致、查询不到 Owner 等。先写“测试意图”，再考虑测试代码。

测试名称	前置条件	操作	期望结果	为什么重要
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____

阶段验收

能否从一个测试用例反向说明“这个功能的契约是什么”？如果只能解释 MockMvc 语法，还没有完成本实训的核心目标。

实训 7 GitHub 代码导航与 Issue 分析

7.1 定义与引用：比全文搜索更可靠

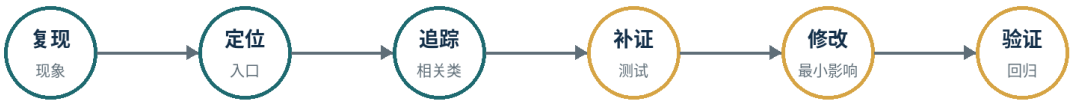
GitHub 官方代码导航支持 Java 等语言的符号浏览、跳转到定义以及查找引用。对于项目理解，这意味着我们可以围绕“符号”而不是“字符串”追踪一个类或方法在哪里定义、被谁使用，从而减少同名文本造成的干扰。

操作	适合回答的问题
Go to definition	这个方法/类真正在哪里实现？
Find references	谁依赖或调用了它？
Symbols	当前文件有哪些重要类/方法？
History / Blame	这段代码为何、何时发生变化？

7.2 从真实 Issue 进入维护场景

一个好的开源实训不应该停在“阅读已有设计”，还应进入“发现问题—定位代码—验证影响”的维护过程。2026 年仓库曾出现关于 Owner 与 Pet 名称校验的 Issue，问题描述指出纯数字、特殊字符或过长名称可能被接受，并将 NamedEntity、Owner、Pet 等列为受影响位置。这个例子非常适合训练从业务现象反推公共抽象。

从 Issue 到代码：真实维护任务的最小闭环



示例：名称校验问题

Issue → NamedEntity / Owner / Pet → 验证规则 → 测试用例 → 回归

图 7-1 从 Issue 到代码的最小维护闭环

7.3 Issue 分析卡

项目	填写内容
现象	用户观察到什么？
复现	最短复现步骤是什么？
入口	哪个页面/请求首先相关？
公共抽象	问题是否集中在共同父类或公共验证？
受影响类	哪些类直接/间接受影响？
已有测试	哪些测试可用于回归？
最小修改	怎样减少修改范围？
新增测试	怎样防止问题再次出现？

注意

Issue 中给出的“可能修复方案”本身也不是最终真相。它是问题提出者的建议，仍需结合当前代码、维护者讨论和测试判断是否合适。

7.4 拓展：读一次代码历史

任选一个你已经理解的类，查看其提交历史或 blame 信息。不要追求读完历史，而是选择一次有意义的修改，回答：修改前后职责是否变化？测试是否同时变化？提交信息能否解释设计动机？这能帮助学生认识到代码结构不是静态答案，而是长期演化结果。

综合实训 输出项目理解报告

8.1 综合任务说明

你将以 Spring PetClinic 或教师指定的同规模开源项目为对象，完成一份“项目理解报告”。报告不是 README 摘抄，也不是 AI 对话整理，而是把项目压缩成几个可以被他人复用的模型，并为重要结论附上证据。

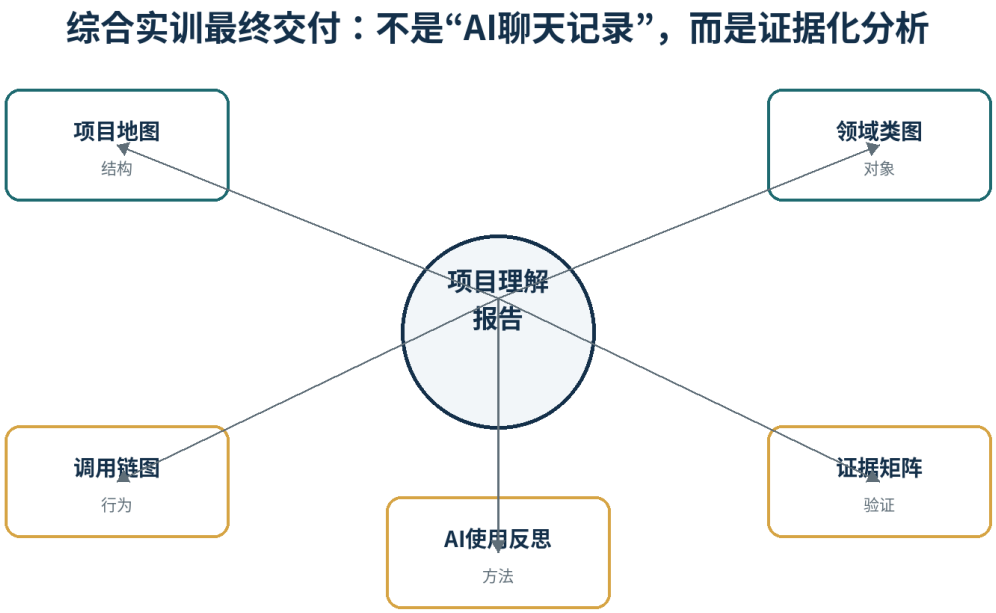


图 8-1 综合实训最终交付物

8.2 必交成果

1. 一页项目地图：根目录、核心包、配置、测试和入口。
2. 一张领域类图：至少包含 5 个核心类，并区分继承与关联。
3. 一张业务调用链图：从用户动作到数据访问再到返回。
4. 一张证据矩阵：至少记录 8 个重要判断及其证据。
5. 一页 AI 使用反思：至少记录 2 个 AI 帮助显著的地方和 1 个被代码纠正的地方。

8.3 项目理解报告结构

章节	建议内容	建议篇幅
1 项目概述	用途、技术栈、启动方式	1 页

章节	建议内容	建议篇幅
2 项目结构	目录地图与模块职责	1–2 页
3 领域对象	类职责、继承与关联	2 页
4 动态行为	一个典型业务调用链	2 页
5 测试与证据	测试覆盖、证据矩阵	1–2 页
6 Issue 或维护任务	问题—代码—测试闭环	1 页
7 AI 使用反思	有效提问、错误与修正	1 页

8.4 评分量表

评价维度	优秀	合格	需改进	分值
结构分析	能说明优先级与模块边界	能列出主要目录	只罗列文件	15
对象关系	关系准确且有证据	主要关系基本正确	箭头含义混乱	15
调用链	按实际执行顺序且可验证	主流程基本完整	以猜测代替追踪	20
测试理解	能从断言反推契约	能解释测试目的	只翻译语法	15
证据意识	关键结论都有证据	大部分有证据	以 AI 答案为依据	15
AI 使用	能提问、反证、修正	能使用 AI 辅助	直接复制 AI 输出	10
表达与图示	图表清晰、可复用	基本清晰	结构杂乱	10

高分标准

高分不取决于“使用了多少 AI”，而取决于你是否能把 AI 输出转化为可验证的项目知识。

8.5 口头答辩问题

1. 如果删掉某个核心类，最先坏掉的功能是什么？你的依据是什么？

2. 你画的类图里，哪一条关系最容易被误判？
3. 你的调用链中，哪一步是框架完成、哪一步是项目代码完成？
4. AI 在哪个问题上给过你看似合理但实际不准确的答案？
5. 如果项目升级后目录变化，你的分析方法中哪些部分仍然有效？

附录 A AI 提示词卡

以下提示词不是“标准答案生成器”，而是用于改变 AI 的工作方式：限定范围、要求证据、允许不确定、鼓励反证。教师可将本页单独打印给学生。

A1 仓库地图

只分析目录、构建、入口、配置和测试，不进入方法细节。给出阅读优先级，并说明每个判断的验证位置。

A2 类责任

从业务对象/责任、状态、行为、关系、删除影响五个维度分析该类；每点给出代码证据。

A3 调用链

围绕一个具体用户动作，按执行顺序追踪；不确定的步骤明确标记待验证。

A4 反证

请指出你刚才解释中最可能错误的三点，并告诉我如何在当前仓库中验证。

A5 比较设计

比较当前实现与另一种实现，但先描述“当前代码事实”，再讨论优缺点。

A6 测试归纳

把测试按场景分组，说明输入、动作、断言，以及这些断言揭示的职责。

A7 影响分析

如果修改这个字段/方法，列出可能受影响的调用者、测试、页面和数据关系，并标注证据。

A8 Issue 定位

根据复现步骤寻找最可能的入口类、公共抽象、验证位置和测试位置；不要直接给修复代码。

A9 术语解释

用当前项目中的具体类解释这个框架术语，不要只给通用定义。

A10 学习检查

根据我刚才的项目分析，出 5 个必须回到代码才能回答的问题，检验我是否真正理解。

附录 B 学生实训记录模板

B.1 仓库观察表

观察对象	第一次判断	证据	修正后的理解
对象 1	_____	_____	_____
对象 2	_____	_____	_____
对象 3	_____	_____	_____
对象 4	_____	_____	_____
对象 5	_____	_____	_____
对象 6	_____	_____	_____

B.2 核心类卡

类名	职责	关键状态	关键方法	关系	证据
类 1	_____	_____	_____	_____	_____
类 2	_____	_____	_____	_____	_____
类 3	_____	_____	_____	_____	_____
类 4	_____	_____	_____	_____	_____
类 5	_____	_____	_____	_____	_____
类 6	_____	_____	_____	_____	_____

B.3 AI 证据矩阵

AI 结论	证据类型	证据位置	结果	备注
结论 1	A/B/C/D	_____	确认/修正	_____

AI 结论	证据类型	证据位置	结果	备注
结论 2	A/B/C/D	_____	确认/修正	_____
结论 3	A/B/C/D	_____	确认/修正	_____
结论 4	A/B/C/D	_____	确认/修正	_____
结论 5	A/B/C/D	_____	确认/修正	_____
结论 6	A/B/C/D	_____	确认/修正	_____
结论 7	A/B/C/D	_____	确认/修正	_____
结论 8	A/B/C/D	_____	确认/修正	_____

B.4 调用链记录

序号	触发	执行者	方法/动作	数据变化	下一步
1	_____	_____	_____	_____	_____
2	_____	_____	_____	_____	_____
3	_____	_____	_____	_____	_____
4	_____	_____	_____	_____	_____
5	_____	_____	_____	_____	_____
6	_____	_____	_____	_____	_____
7	_____	_____	_____	_____	_____
8	_____	_____	_____	_____	_____

B.5 实训反思

今天最重要的新理解是什么？

_____

_____

_____

AI 在哪一步最有帮助？为什么？

_____

_____

_____

AI 在哪一步误导或过度推断？你如何发现？

_____

_____

_____

如果换一个完全陌生的项目，你准备保留哪三步方法？

_____

_____

_____

附录 C 教师参考与教学组织建议

C.1 教学重点

本教材最容易被误教成“Spring Boot 入门”。教师需要持续把注意力拉回“代码理解方法”：为什么先读这个文件、为什么这条关系成立、为什么这个结论可信。框架知识只在帮助理解当前项目时讲解，不要求学生在一次实训中掌握 Spring 的全部机制。

C.2 课堂节奏建议（8 学时版本）

课时	课堂活动	教师动作	学生产出
1	项目运行 + 仓库侦察	示范 5 分钟扫描	项目地图草图
2	入口与技术栈	演示“AI 标准分层幻觉”	启动链 + 纠错记录
3–4	领域对象与类关系	追问箭头含义	领域类图
5–6	新增 Owner 调用链	断点演示 + 证据核验	调用链图
7	测试与 Issue	从失败路径反推职责	测试卡 / Issue 卡
8	汇报与互评	用证据追问	综合报告框架

C.3 常见学生误区与纠正

误区	课堂表现	纠正策略
逐文件阅读	进度慢且无法总结	强制先交一页地图
把包结构当架构	看到目录就下结论	要求用实际依赖核验
所有箭头都写“调用”	静态关系与动态行为混淆	分开画类图和调用链
AI 答案直接入报告	没有证据位置	每条结论附证据等级
只看成功路径	忽略验证和异常	至少分析一个失败测试
追求复杂设计模式	“什么都是模式”	只接受有参与类与问题背景的模式判断

C.4 教师参考答案（核心方向）

- 仓库入口：README、构建文件、src/main、src/test、resources 是第一轮高价值位置。
- 启动入口：PetClinicApplication；main 方法将应用启动交给 Spring Boot。
- OwnerController 的新增 Owner 流程直接依赖 OwnerRepository；不要自动补出独立 Service 层。
- 领域理解应优先关注 BaseEntity / NamedEntity / Person 的共性抽象，以及 Owner—Pet—Visit 等关联。
- 测试不只是“覆盖率材料”，而是理解输入、输出、错误和回归边界的证据。
- AI 作为 D 级证据使用；源码/配置和测试/运行优先级更高。

建议的追问方式

学生汇报时不要先问“答案对不对”，而问：“你从哪里知道？”、“如果我说相反，哪一段代码能证明你？”、“这个箭头是静态关系还是运行时调用？”

参考资料与版本说明

本教材中的项目事实以 2026 年 9 月公开可访问的官方主仓库与官方文档为主要依据。由于开源项目持续更新，类名、依赖版本、Issue 状态和目录结构可能发生变化；课堂使用时应优先检查当前仓库。

1. spring-projects/spring-petclinic. Spring PetClinic Sample Application, GitHub official repository. Accessed 2026-09-23.

2. Spring PetClinic README. Run locally, database configuration, IDE usage and main class guidance. Accessed 2026-09-23.

3. Spring PetClinic pom.xml. Current build metadata and Java version setting. Accessed 2026-09-23.

4. Spring PetClinic OwnerController.java. Owner request handling and repository dependency. Accessed 2026-09-23.

5. Spring PetClinic OwnerControllerTests.java. Controller behavior tests. Accessed 2026-09-23.

6. GitHub Docs. Navigating code on GitHub: symbols, definitions and references. Accessed 2026-09-23.

7. Spring PetClinic Issue #2554. Owner/Pet name validation discussion (used as maintenance-analysis example).

开源与教学使用说明

Spring PetClinic 主仓库采用 Apache License 2.0。本教材未复制大段项目源代码，而以文件路径、类名、方法职责、简化结构图与教学化伪代码为主，用于说明程序理解方法。学生提交报告时也建议优先引用代码位置和自己的结构图，而不是大段粘贴源文件。

结语

AI 正在显著降低“进入陌生代码”的成本，但没有降低判断的价值。真正可靠的程序理解，不是能够快速生成一段解释，而是能够把解释重新连接到代码、测试和运行事实。面向对象知识在这里不再只是写类的规则，而成为压缩复杂软件、理解职责边界和追踪变化影响的语言。

> **一句话带走**
>
> AI 让你更快提出“这个系统可能怎样工作”；工程证据决定“它实际上怎样工作”。